Session 02 - The Caesar Cipher as Functions - Tests

Python Contoso Vault - Session 02 of 07

Part 1 - Core part A: shift.py

Goal

Read a message and a key. Encrypt the message, then decrypt it again to prove the round trip works. Keep the case of each letter. Leave every character that is not a letter unchanged.

Run `python shift.py` in the terminal. Type each input line and press Enter after it.

Instructions

The class builds steps 1 to 10 for capital letters only, then commits the file. Finish at home: make steps 3, 4, 7 and 8 work for lowercase letters too.

1. Read a message as a string.
2. Read a key from 0 to 25 as an integer.
3. Shift each letter of the message forward by the key, within its own case.
4. Wrap any shifted letter that passes the end of its alphabet.
5. Leave every character that is not a letter unchanged.
6. Print the ciphertext.
7. Shift each letter of the ciphertext back by the key.
8. Wrap any shifted letter that passes the start of its alphabet.
9. Print the plaintext.
10. Prove that the plaintext matches the message.

Constraints

- Read each value with a prompt on the same line, as in `input("Message: ")`. The prompts print in this order: `"Message: "` for the message, then `"Key: "` for the key. Each prompt is its label, a colon and one trailing space. Type each prompt exactly as shown. A program with different prompt text does not meet the test plan.
- After the two prompts, the output is exactly two lines: `Ciphertext: <text>`, then `Plaintext: <text>`. Put one space after each colon.

Design

The following rules say how to write the code. Follow them.

- Wrap shifted letters with an `if`, without `%`.
- Prove the round trip with an `assert`.
- Give every variable a type hint.

Input validation

Not required. Assume the key is always a whole number from 0 to 25. Any message is valid, including an empty one.

Test plan

The program prints both prompts first, so the expected output in the following test plan shows the two lines printed after the prompts.

Test	Message	Key	Expected output	Actual output	Pass
1	`Meet at Contoso HQ!`	`3`	`Ciphertext: Phhw dw Frqwrvr KT!` `Plaintext: Meet at Contoso HQ!`		
2	`Zebra zone 42, room 7.`	`25`	`Ciphertext: Ydaqz ynmd 42, qnnl 7.` `Plaintext: Zebra zone 42, room 7.`		
3	`Contoso HQ`	`0`	`Ciphertext: Contoso HQ` `Plaintext: Contoso HQ`		
4	`Contoso HQ`	`1`	`Ciphertext: Dpouptp IR` `Plaintext: Contoso HQ`		
5	`Contoso HQ`	`13`	`Ciphertext: Pbagbfb UD` `Plaintext: Contoso HQ`		
6	`Contoso HQ`	`25`	`Ciphertext: Bnmsnrn GP` `Plaintext: Contoso HQ`		
7	`2026-11-01, 9:00!`	`7`	`Ciphertext: 2026-11-01, 9:00!` `Plaintext: 2026-11-01, 9:00!`		
8	`mIxEd CaSe`	`4`	`Ciphertext: qMbIh GeWi` `Plaintext: mIxEd CaSe`		
9	empty line	`5`	`Ciphertext:` `Plaintext:`		

Done criteria

- ☐ Every test plan row prints exactly the expected lines.
- ☐ The script runs from the terminal with `python shift.py`.

Part 2 - Core part B: caesar.py

Goal

Turn the cipher of `shift.py` into functions. Make `encrypt` and `decrypt` work with any whole-number key. Make the file safe to import, and make it check its own round trip when it runs.

Run `python caesar.py` in the terminal. To try one function, open the Python shell in the `02-caesar-cipher-as-functions` folder and run `from caesar import decrypt, encrypt, shift_char`.

Instructions

The class builds steps 1 and 2 for keys 0 to 25, using an `if` for the wrap. Finish at home: add step 1.1 to `shift_char`, then step 3 and step 4.

1. Define a function `shift_char` that takes `character` as a string and `key` as an integer.
   1.1. Turn the key into the key from 0 to 25 that gives the same result as the original key. For example, 27 becomes 1 and −1 becomes 25.
   1.2. Shift `character` forward by the key, within its own case, when it is a letter.
   1.3. Leave `character` unchanged when it is not a letter.
   1.4. Return the resulting character as a string.
2. Define a function `encrypt` that takes `text` as a string and `key` as an integer.
   2.1. Shift each character of `text` forward by the key.
   2.2. Return the ciphertext as a string.
3. Define a function `decrypt` that takes `text` as a string and `key` as an integer.
   3.1. Decrypt `text` by encrypting it with the opposite key.
   3.2. Return the plaintext as a string.
4. When the file runs as a program:
   4.1. Prove that decrypting an encrypted text returns the original text.

Constraints

- Any whole-number key works, including negative and very large keys. Two keys that differ by 26 give the same ciphertext.
- Only the 26 letters of the alphabet shift, in uppercase and lowercase. Each keeps its case. Every other character stays unchanged.
- The three functions return their result. None of them prints or reads input.
- Importing the file reads no input and prints nothing.
- Run as a program, the file reads no input, prints nothing and ends without an error.

Design

The following rules say how to write the code. Follow them.

- `encrypt` calls `shift_char` for each character.
- Reduce the key with `%`, without an `if` wrap.
- Put the code that runs as a program under `if __name__ == "__main__":`.
- Prove the round trip with an `assert`.
- Give every variable and function a type hint.

Input validation

Not required. Assume every text uses English letters, digits, spaces and punctuation only. Any key is a whole number. An empty text is valid.

Test plan

Test	Call or run	Expected output	Actual output	Pass
1	`encrypt("Meet at Contoso HQ!", 3)`	`"Phhw dw Frqwrvr KT!"`		
2	`encrypt("Contoso HQ", 29)`	`"Frqwrvr KT"`		
3	`encrypt("Contoso HQ", -1)`	`"Bnmsnrn GP"`		
4	`encrypt("Contoso HQ", 26)`	`"Contoso HQ"`		
5	`encrypt("Contoso HQ", 10**9)`	`"Oazfaea TC"`		
6	`encrypt("mIxEd CaSe", -30)`	`"iEtAz YwOa"`		
7	`encrypt("Rs 2,500 for the Kandy guest house.", 5)`	`"Wx 2,500 ktw ymj Pfsid lzjxy mtzxj."`		
8	`encrypt("", 7)`	`""`		
9	`decrypt("Phhw dw Frqwrvr KT!", 3)`	`"Meet at Contoso HQ!"`		
10	`decrypt("Contoso HQ", -3)`	`"Frqwrvr KT"`		
11	`decrypt("Contoso HQ", 10**9)`	`"Qcbhcgc VE"`		
12	`shift_char("c", 3)`	`"f"`		
13	`shift_char("Z", 1)`	`"A"`		
14	`shift_char(" ", 5)`	`" "`		
15	`import caesar`	Reads no input and prints nothing.		
16	`python caesar.py`	Reads no input, prints nothing, ends without an error.		

Done criteria

- ☐ Every test plan row gives exactly the expected result.
- ☐ The script runs from the terminal with `python caesar.py`.

Part 3 - Challenge: best_guess.py

Goal

Find the most likely key for a ciphertext. For each key, decrypt the ciphertext and compare the letter counts of the result with English text.

Keep `caesar.py` and `english.py` in the same folder as `best_guess.py`. Open the Python shell there, run `from best_guess import best_guess`, then call `best_guess` with each ciphertext in the test plan.

Provided

`english.py` arrives in this folder when you sync your fork. It defines `expected_freq(letter: str) -> float`. It takes one lowercase letter and returns its share of the letters in English text.

The shares count every letter of Jane Austen's *Pride and Prejudice*, from Project Gutenberg. The count is case-insensitive.

The following table lists the share of each letter in English text, as `expected_freq` returns it.

Letter	Share	Letter	Share	Letter	Share	Letter	Share
a	0.07771	h	0.06319	o	0.07452	v	0.01065
b	0.01683	i	0.0711	p	0.01544	w	0.02257
c	0.02523	j	0.00163	q	0.00115	x	0.00178
d	0.04111	k	0.00595	r	0.06025	y	0.02348
e	0.12871	l	0.04079	s	0.06196	z	0.00172
f	0.02237	m	0.02727	t	0.08749		
g	0.01878	n	0.07031	u	0.02799		

Instructions

1. Import `decrypt` from `caesar.py` and `expected_freq` from `english.py`.
2. Define a function `best_guess` that takes a ciphertext as a string.
   2.1. Count how many characters of the ciphertext are letters, as an integer. Uppercase and lowercase letters both count. Spaces, digits and punctuation do not.
   2.2. Return 0 if the ciphertext has no letters.
   2.3. Start the best key at 0, as an integer.
   2.4. Start the best score at infinity, as a float.
   2.5. Try each key from 0 to 25, as an integer, in turn:
      2.5.1. Decrypt the ciphertext with the current key.
      2.5.2. Start the key's score at 0, as a float.
      2.5.3. Go through each of the 26 letters of the alphabet, from a to z, in turn:
         2.5.3.1. Count how many times the letter appears in the plaintext, as an integer. The count is case-insensitive, so `A` and `a` both count as the same letter.
         2.5.3.2. Work out the expected count, as a float: the number of letters in the ciphertext times the letter's share from `expected_freq`.
         2.5.3.3. Add (count - expected count) squared, divided by the expected count, to the key's score.
      2.5.4. Make the key the best key, and its score the best score, if its score is lower than the best score.
   2.6. Return the best key as an integer.

Constraints

- The returned key decrypts the ciphertext.
- When two keys tie, the smaller key wins.
- The function returns its result. It neither prints nor reads input.

Design

The following rules say how to write the code. Follow them.

- Give every variable and function a type hint.

Input validation

Not required. Any ciphertext is valid, including an empty one.

Test plan

Test	Ciphertext	Letters in ciphertext	Expected output	Actual output	Pass
1	`Gsrxsws wlst xieq: xli ria tvmgi pmwx jsv xli Gspsqfs wxsvi wxevxw sr Qsrhec. Izivc mxiq yrhiv jmzi lyrhvih vytiiw qsziw xs xli jvsrx wlipj.`	113	`4`		
2	`Kyv Tfekfjf kirmvc uvjb sffbj kirzej, sljvj reu xlvjk yfljvj wfi jkrww. Jveu kyv urkvj fw pfli kizg reu kyv kfne pfl nzcc mzjzk kf kyv uvjb.`	111	`17`		
3	`Zlkqlpl pqlzh obmloq: qeb Hxkav txobelrpb eliap qeobb qelrpxka yxdp, kfkb erkaoba ylqqibp lc txqbo xka qtl erkaoba ylubp lc qbx clo qeb pelmp.`	116	`23`		
4	`LXWCXBX YJHAXUU WXCRLN: BJUJARNB OXA WXENVKNA ANJLQ NENAH KJWT JLLXDWC XW CQN CFNWCH OROCQ. LXWCJLC CQN ORWJWLN CNJV FRCQ JWH ZDNBCRXW JKXDC HXDA YJHBURY.`	128	`9`		
5	`Contoso shop team: the new price list for the Colombo store starts on Monday. Every item under five hundred rupees moves to the front shelf.`	113	`0`		
6	empty string	0	`0`		
7	`2026-11-08!`	0	`0`		
8	`Cvaa pgg rvjdl` (true key 1)	12	`15`		

Test 8 shows the scoring method failing on a short ciphertext. It returns 15, but the true key is 1.

Done criteria

- [ ] Every test plan row returns exactly the expected key.

Marking

This take-home is marked out of 100, with a pass mark of 60.

Part	Tests	Design	Total
Core	64	16	80
Challenge	16	4	20
Total	**80**	**20**	**100**

Guidelines

- Test marks show that the program works. The instructor's tests run it on every row of the test plan, plus further cases.
- Design marks show that the code is built well. The instructor reads the code against each Design rule.
- A pass needs 60 marks from the Core alone. Challenge marks count towards the total once the Core reaches 60.
- Marking uses the last commit pushed by 11:59 PM on the day before the next session.
- The submission is one GitHub link to the fork, posted on the portal.